

TUALI GROWTH AGENT

Documentación del Código, Funcionamiento, Pruebas y Lecciones del Desarrollo

Proyecto Hack4Her | Ecosistema Tuali - Arca Continental | Versión 1.0

Fecha: Junio 2026

Alcance del documento. Este documento consolida la documentación técnica del agente desarrollado, explica su funcionamiento interno, resume los usos de prueba realizados, enumera los problemas encontrados durante el desarrollo y registra las configuraciones de seguridad consideradas. La documentación se basa en los archivos Python compartidos, la estructura del proyecto y el dossier oficial del reto Tuali Growth.

Contenido

1. Resumen técnico del proyecto
2. Estructura del proyecto
3. Arquitectura funcional
4. Flujo de ejecución del agente
5. Documentación por módulo
6. Modelo de datos y colecciones MongoDB
7. Pruebas realizadas y archivos de test
8. Casos de uso probados
9. Problemas encontrados durante el desarrollo
10. Configuraciones y controles de seguridad
11. Limitaciones actuales
12. Recomendaciones para cierre del hackathon

1. Resumen técnico del proyecto

Tuali Growth Agent es un agente de crecimiento comercial diseñado para generar recomendaciones personalizadas dentro del ecosistema Tuali. El objetivo técnico del agente es recibir contexto del cliente, calcular oportunidades de crecimiento, presentarlas con lenguaje claro mediante IA generativa, registrar la retroalimentación del usuario y ajustar futuras recomendaciones con base en aprendizaje.

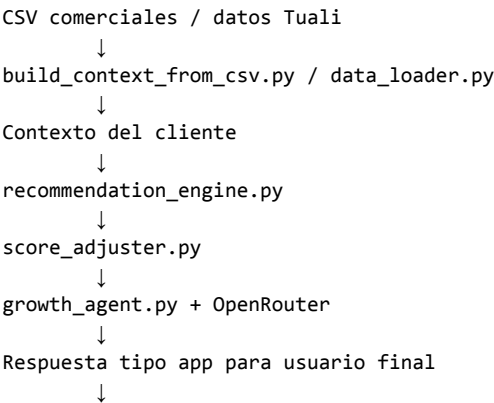
Elemento	Descripción
Tipo de solución	Motor de recomendaciones + capa de IA generativa + aprendizaje por feedback.
Usuario objetivo	Cliente detallista que utiliza Tuali para comprar productos de Arca Continental.
Entrada principal	Historial de compras, productos frecuentes, oportunidades por CEDIS, promociones y loyalty.
Salida principal	Recomendaciones accionables: reabastecimiento, oportunidad CEDIS y promoción.
Persistencia	MongoDB: tableau_recommendations, feedback y learning_summary.
Visualización	CSV/Tableau y prototipo de interfaz tipo app.

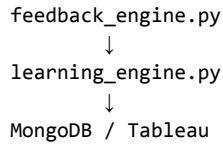
2. Estructura del proyecto

La carpeta local del proyecto contiene módulos de motor de recomendaciones, aprendizaje, persistencia, carga de datos y pruebas. Los archivos más relevantes son:

Archivo / carpeta	Rol dentro del proyecto
growth_agent.py	Capa de IA generativa. Convierte recomendaciones calculadas en mensajes claros para la app.
recommendation_engine.py	Motor de reglas. Calcula oportunidades de reabastecimiento, CEDIS y promociones.
feedback_engine.py	Registra respuestas del usuario y actualiza recomendaciones existentes.
learning_engine.py	Resume feedback histórico y calcula tasa de aceptación por producto/tipo.
score_adjuster.py	Ajusta el score de recomendaciones usando aprendizaje previo.
save_to_mongo.py	Guarda recomendaciones en MongoDB para consumo de Tableau.
data_loader.py	Carga de datos desde fuentes CSV.
build_context_from_csv.py	Construcción de contexto del cliente desde CSV comerciales.
export_tableau_csv.py	Exportación de recomendaciones a CSV para Tableau.
safe_context.py	Preparación/filtrado de contexto seguro para el agente.
data/	Datasets base: pedidos, métricas cliente-producto, popularidad y tendencias.
tests/	Pruebas unitarias y funcionales del MVP.

3. Arquitectura funcional





Principio de diseño. El modelo generativo no decide qué recomendar. La decisión se calcula primero en el motor de recomendaciones. La IA únicamente redacta la salida de manera entendible y personalizada según la protopersona.

4. Flujo de ejecución del agente

Paso	Etapas	Descripción
1	Entrada de contexto	Se recibe un diccionario con cliente_id, alias, CEDIS, historial, promociones, loyalty y pedido sugerido.
2	Cálculo de oportunidades	recommendation_engine.py analiza historial, productos frecuentes, oportunidades CEDIS y promociones.
3	Ajuste por aprendizaje	score_adjuster.py puede modificar la prioridad de un producto según aceptación o rechazo previo.
4	Generación de mensaje	growth_agent.py arma un prompt con reglas estrictas contra alucinaciones y llama a OpenRouter.
5	Presentación al usuario	El resultado se muestra como pantalla de app con acciones: Agregar / No me interesa.
6	Registro de feedback	feedback_engine.py guarda la respuesta en MongoDB y actualiza tableau_recommendations.
7	Aprendizaje	learning_engine.py calcula tasa de aceptación y genera learning_summary.

5. Documentación por módulo

5.1 recommendation_engine.py

Contiene la función generar_oportunidades(contexto). Calcula recomendaciones de reabastecimiento, oportunidad CEDIS y promoción. Usa Counter para identificar productos comprados con mayor frecuencia y calcula scores normalizados. También integra ajuste de score por aprendizaje.

Elemento	Detalle
Funciones clave	calcular_score(valor, maximo), generar_oportunidades(contexto)
Salida / efecto	Devuelve un diccionario con cliente_alias, cliente_id, protopersona, CEDIS, total_compras, recomendaciones, loyalty, pedido_sugerido, productos_validos y timestamp.

5.2 growth_agent.py

Capa de presentación inteligente. Usa OpenAI SDK apuntando a OpenRouter. El modelo configurado es meta-llama/llama-3.1-8b-instruct. Construye un prompt con reglas de comunicación y restricciones explícitas para no inventar datos.

Elemento	Detalle
Funciones clave	generar_recomendacion(contexto_cliente)
Salida / efecto	Devuelve texto listo para una pantalla tipo app móvil con secciones de oportunidades, puntos y productos recomendados.

5.3 feedback_engine.py

Registra la acción del usuario frente a una recomendación. Inserta un documento en feedback y actualiza el registro correspondiente en tableau_recommendations.

Elemento	Detalle
Funciones clave	guardar_feedback(cliente_id, producto, tipo_recomendacion, respuesta)
Salida / efecto	Guarda producto, tipo_recomendacion, respuesta y timestamp.

5.4 learning_engine.py

Procesa todo el histórico de feedback. Agrupa por producto y tipo de recomendación. Calcula tasa de aceptación y clasifica el aprendizaje como alta, media o baja aceptación.

Elemento	Detalle
Funciones clave	calcular_aprendizaje()
Salida / efecto	Sobrescribe learning_summary con el resumen actualizado.

5.5 save_to_mongo.py

Ejecuta generar_oportunidades(contexto_cliente), transforma cada recomendación en documentos orientados a Tableau y los guarda en MongoDB.

Elemento	Detalle
Funciones clave	guardar_recomendaciones_tableau(contexto_cliente)
Salida / efecto	Inserta registros en tableau_recommendations con título, evidencia, motivo, predicción, impacto, acción y botones.

6. Modelo de datos y colecciones MongoDB

Colección	Propósito	Campos principales
tableau_recommendations	Recomendaciones finales para visualización y seguimiento.	cliente_id, cliente_nombre, cedis, tipo, producto, score, evidencia, motivo, prediccion, impacto, accion, botones, feedback, timestamp.
feedback	Registro de interacción del usuario.	cliente_id, producto, tipo_recomendacion, respuesta, timestamp.
learning_summary	Resumen de aprendizaje por producto y tipo.	producto, tipo_recomendacion, total_feedback, aceptadas, rechazadas, tasa_aceptacion, aprendizaje, timestamp.

Dataset	Uso esperado
PEDIDOS_MAESTRA.csv	Historial de pedidos. Base para detectar frecuencia, recompra y productos recurrentes.
TB_METRICAS_CLIENTE_PRODUCTO.csv	Métricas por cliente-producto. Ayuda a identificar productos frecuentes y ranking individual.
TB_POPULARIDAD_PRODUCTO.csv	Popularidad general o regional. Ayuda a comparar demanda.
TB_RECOMENDACIONES_TENDENCIA.csv	Tendencias y recomendaciones externas. Apoya oportunidad CEDIS.
tableau_recommendations.csv	Archivo de salida para prototipo en Tableau.

7. Pruebas realizadas y archivos de test

La carpeta tests/ contiene pruebas orientadas a validar conectividad, motores internos, persistencia y salida para Tableau. No todos los archivos de test fueron compartidos con contenido completo; esta sección documenta su propósito por nombre y por el comportamiento observado durante el desarrollo.

Archivo de prueba	Objetivo probable / validado
test_agent.py	Validar creación y ejecución básica del agente.
test_behavior.py	Validar lógica de comportamiento del cliente.
test_env.py	Validar carga de variables de entorno.
test_feedback.py	Validar registro de respuestas Agregar / No me interesa.
test_key.py	Validar disponibilidad de API key.
test_learning.py	Validar cálculo de aprendizaje a partir de feedback.
test_mongo.py	Validar conexión con MongoDB.
test_mongo_data.py	Validar datos insertados o consultados en MongoDB.
test_openrouter.py	Validar conexión con OpenRouter y modelo de IA.
test_recommendation_engine.py	Validar generación de recomendaciones.
test_score_adjuster.py	Validar ajuste de score por aprendizaje.
test_tableau_records.py	Validar estructura de registros exportables a Tableau.

7.1 Pruebas funcionales observadas

Prueba	Resultado esperado
Ejecución de test_growth_ui.py	Mostrar oportunidades para crecer en consola: reabastecimiento y nuevos productos.
Prueba con perfil Fernanda	El mensaje debe hablar directamente a la usuaria y sonar como pantalla de app.
Prueba de recomendaciones reales	Usar productos calculados desde datos reales, no sample data.
Prueba MongoDB	Guardar recomendaciones en collection tableau_recommendations.
Prueba feedback	Actualizar feedback en la recomendación y registrar histórico en feedback.
Prueba aprendizaje	Generar learning_summary con tasa de aceptación y aprendizaje textual.
Prueba Tableau	Generar o exportar registros consumibles por dashboard.

8. Casos de uso probados

Caso de uso	Descripción	Resultado esperado
Reabastecimiento de producto frecuente	Cliente con historial repetido de Coca-Cola.	Recomendación: agregar Coca-Cola por alta necesidad de reposición.
Oportunidad por CEDIS	Producto con buen comportamiento en negocios similares de zona/CEDIS.	Recomendación: probar pocas unidades con bajo riesgo.
Promoción disponible	Producto con promoción activa.	Recomendación: aprovechar promoción para incrementar ticket promedio.
Loyalty / puntos	Cliente con puntos actuales o regla de acumulación.	Mostrar sección de recompensas y oportunidad de puntos.
Feedback positivo	Usuario presiona Agregar.	Aumenta aceptación histórica y puede elevar prioridad futura.
Feedback negativo	Usuario presiona No me interesa.	Reduce prioridad futura si la tasa de aceptación es baja.
Salida para Tableau	Recomendaciones guardadas en MongoDB o CSV.	Dashboard puede mostrar evidencia, score, acción y estado de feedback.

9. Problemas encontrados durante el desarrollo

Problema	Causa probable	Solución aplicada / recomendada
Uso inicial de sample data	MongoDB Atlas Data Federation sólo mostraba opción con datos de ejemplo o requería proveedor.	Se decidió levantar manualmente la fuente y preparar la conexión con datos reales.
Nombre inválido de Data Federation	Se usó underscore en tuali_growth_federation; Atlas sólo aceptaba letras, números y guiones.	Renombrar como tuali-growth-federation.
Necesidad de proveedor en Data Federation	Atlas requería seleccionar proveedor cloud antes de crear federación.	Configurar proveedor permitido o continuar con carga directa a MongoDB.
Riesgo de alucinación de IA	El modelo podía inventar productos, descuentos o impactos si el prompt era abierto.	Se agregaron reglas estrictas: no inventar productos, fechas, porcentajes, descuentos ni impactos.

Interfaz debía parecer app para Fernanda	La salida inicial era más técnica o genérica.	Se rediseñó el prompt para segunda persona y formato de pantalla móvil.
Diferencia entre pedido_sugerido y recomendaciones	El modelo podía listar productos de pedido_sugerido no calculados como recomendación.	Se incluyó lista productos_recomendados y regla de usar sólo esos productos.
Persistencia aún en fase MVP	mongodb.py y app.py aparecen vacíos en la estructura del proyecto.	Consolidar funciones de conexión y crear capa de aplicación real.
Borrado de colecciones/resúmenes	learning_engine usa delete_many({}) para recalculer learning_summary.	Para producción, usar versionado, upsert y conservar histórico.
Archivos grandes CSV	Datasets superiores a cientos de MB pueden afectar memoria y tiempo.	Optimizar lectura por chunks, índices, preagregados o carga a MongoDB.
Dependencia de APIs externas	OpenRouter requiere API key y disponibilidad de red.	Agregar manejo de errores, fallback y timeout.

10. Configuraciones y controles de seguridad

El proyecto incorpora separación básica de configuración mediante .env, .env.example y config.py. Esto evita hardcodear secretos dentro del código fuente, siempre que .env no se comparta ni se suba a repositorios.

Control	Estado	Observación
Variables de entorno	Implementado	OPENROUTER_API_KEY, MONGO_URI y DB_NAME deben leerse desde config.py.
.gitignore	Implementado	Debe incluir .env, venv, __pycache__, archivos CSV pesados y salidas temporales.
Separación de credenciales	Parcial	Existe .env.example; nunca debe contener valores reales.
Control antialucinaciones	Implementado en prompt	No inventar productos, descuentos, fechas, porcentajes ni impactos.
MongoDB RBAC	Recomendado	Crear usuario sólo con permisos necesarios para la base del proyecto.
TLS MongoDB	Recomendado	Usar conexión cifrada y validación de certificados.
Auditoría	Recomendado	Registrar cambios de feedback, inserts, updates y errores.
Protección de datos	Recomendado	Evitar exponer datos personales o comerciales innecesarios en Tableau.

11. Limitaciones actuales

- El documento no incluye código de todos los archivos porque sólo se compartieron módulos principales y estructura del proyecto.
- El contenido exacto de tests/ no fue compartido; se documentó por nombres, flujo y comportamiento observado.
- La persistencia tiene características de MVP: algunas colecciones se recalculan o sobrescriben.
- No se observa manejo completo de excepciones para fallos de MongoDB, OpenRouter o archivos CSV.
- No se observa capa de API formal ni interfaz productiva; el prototipo se apoya en consola, CSV, MongoDB y Tableau.

12. Recomendaciones para cierre del hackathon

Prioridad	Acción
Alta	Crear README.md con instalación, variables de entorno, ejecución y demo.
Alta	Subir sólo .env.example, nunca .env.
Alta	Agregar requirements.txt con dependencias reales.
Alta	Crear script demo único: python run_real_agent.py.
Media	Agregar manejo de errores para MongoDB y OpenRouter.

Media	Cambiar delete many por upsert y control de versiones.
Media	Crear datos mock pequeños para demo local sin exponer datasets completos.
Baja	Agregar API con FastAPI o Streamlit si se requiere visualización interactiva.

Anexo A. Ejemplo de flujo de demo

1. Activar entorno virtual:
venv\Scripts\activate
2. Validar variables de entorno:
python tests/test_env.py
3. Probar conexión OpenRouter:
python tests/test_openrouter.py
4. Probar MongoDB:
python tests/test_mongo.py
5. Generar recomendaciones:
python run_real_agent.py
6. Guardar recomendaciones para Tableau:
python save_to_mongo.py
7. Probar interfaz de consola:
python test_growth_ui.py

Anexo B. Criterios de aceptación

Criterio	Resultado esperado
El agente genera máximo productos calculados	No aparecen productos fuera de recomendaciones.
El texto es entendible para comerciante	Segunda persona, sin tecnicismos y formato tipo app.
La recomendación contiene evidencia	Cada producto incluye evidencia, motivo, predicción, impacto y acción.
Feedback se guarda correctamente	Existe registro en feedback y update en tableau_recommendations.
Aprendizaje se actualiza	learning_summary refleja tasa de aceptación.
Tableau puede consumir datos	Registros contienen campos esperados para dashboard.

Fuentes utilizadas

- Estructura del proyecto: archivos.txt y estructura.txt.
- Módulos de código compartidos: feedback_engine.py, growth_agent.py, learning_engine.py, recommendation_engine.py y save_to_mongo.py.
- Dossier oficial del reto: Reto3_Tuali_Growth_dossier(1).pdf.
- Conversación de desarrollo: pruebas de consola, problemas con Data Federation, integración MongoDB/Tableau y rediseño de salida para Fernanda.